

PGR Intelligence - Backend Architecture

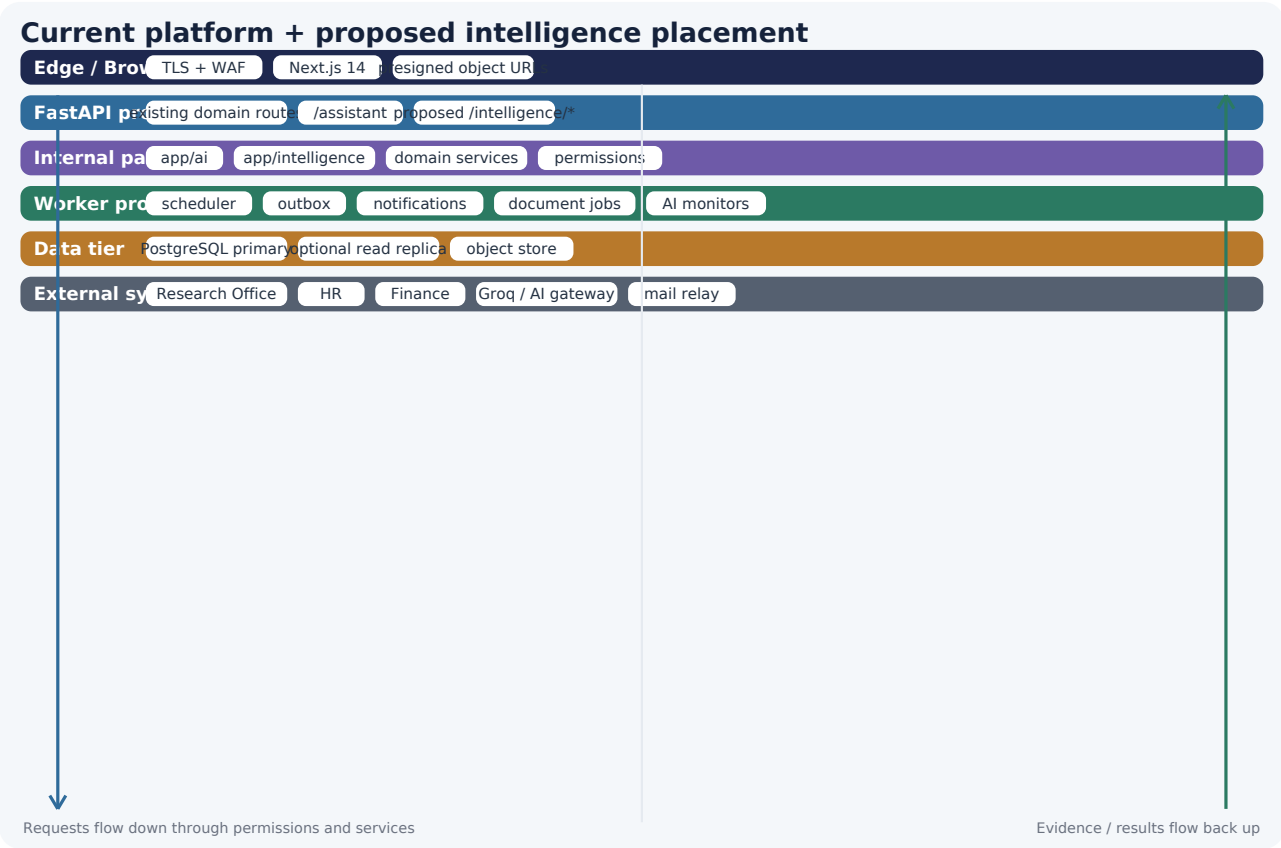
Detailed reference architecture for the proposed AI upgrades, aligned to the existing modular monolith, FastAPI API, PostgreSQL data tier, worker, object store, read-replica option, audit/outbox model and current app/ai contracts.

Backend architecture, data contracts and delivery design	PGR Intelligence Programme
--	----------------------------

Built around the existing PGR platform: lifecycle, supervision, progression, funding, thesis, documents, workflows, Pattern Lab and Ask PGR.

1. Current architecture baseline - preserve it

The supplied deployment design describes a modular monolith with three runnable application processes plus PostgreSQL: Next.js frontend, FastAPI API, APScheduler worker, PostgreSQL primary, optional physical read replica, object storage, mail relay and edge TLS/WAF. The AI upgrade should fit inside this topology rather than creating a separate mesh of autonomous services.



Network alignment: the architecture document currently says egress is limited to SMTP and partner webhook targets. Because your current AI inventory already uses Groq, production egress policy must explicitly include the approved Groq HTTPS endpoint or an internal AI gateway/proxy. Treat this as a required documentation/deployment update.

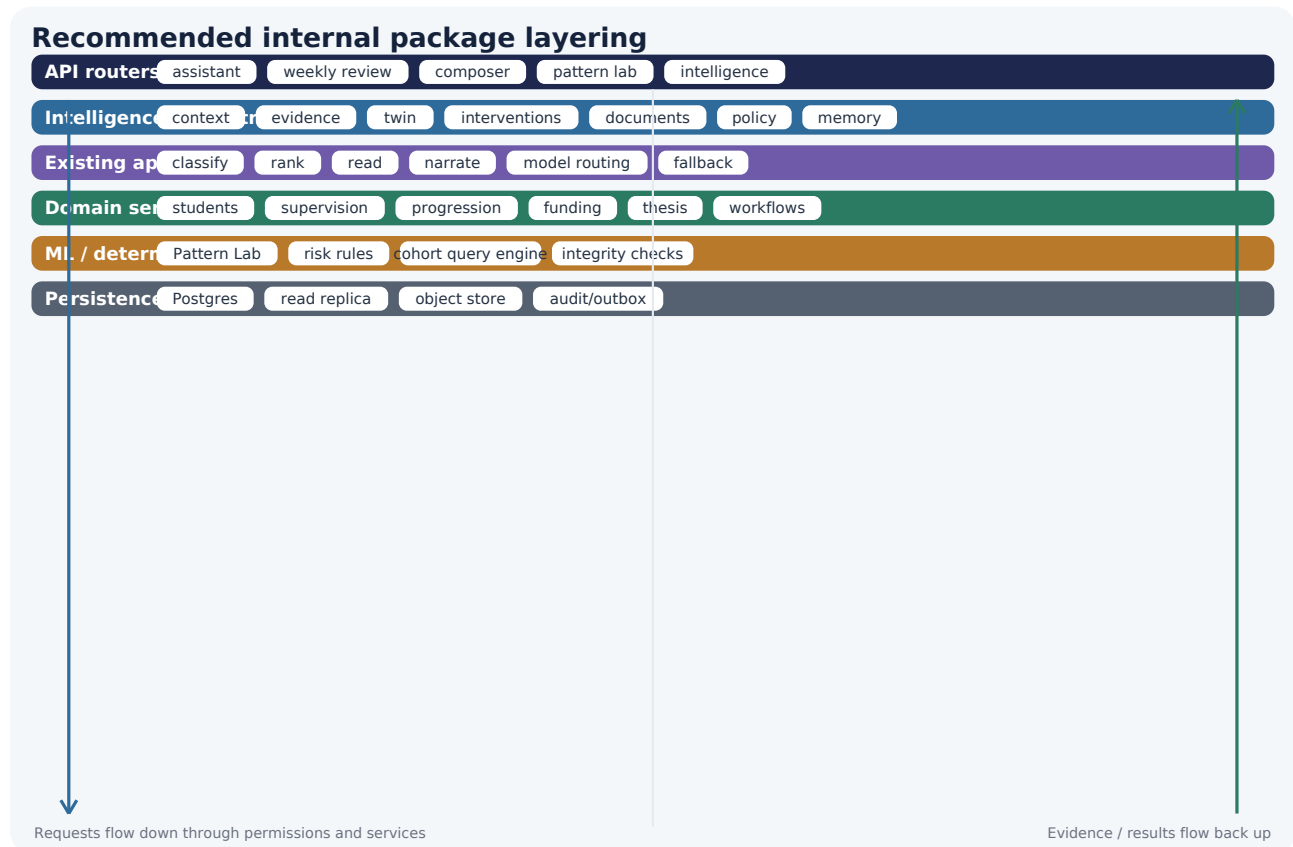
2. Architecture invariants

These rules keep the AI layer aligned with the platform philosophy you already implemented.

Invariant	Implementation meaning
Domain services remain authoritative	No LLM response writes directly to student, funding, milestone, thesis or workflow tables.
Row scoping happens before AI	Context Builder receives only rows/documents authorised for the current principal.
LLM-off path is first-class	Every user-visible capability has a valid deterministic/manual result when Groq is unavailable.
Structured contracts everywhere	app/ai classify/rank/read/narrate and new intelligence objects are schema-validated before use.
Side effects are explicit	AI proposes command drafts; execution goes through existing domain endpoints/services after confirmation.
Evidence is mandatory	Every factual conclusion points to source rows/documents/model outputs and freshness.
Prediction != causal effect	Pattern Lab sensitivity is isolated from deterministic rule simulation and labelled accordingly.

3. Package layering - app/ai stays, app/intelligence is added

Do not expand app/ai until it becomes a second domain layer. Keep it as the low-level Groq/model gateway with your four typed shapes. Add a higher-level app/intelligence package that knows PGR context, evidence, scenarios and intervention orchestration.

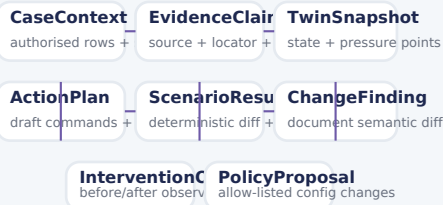


Suggested boundary: app/intelligence can read via domain/query services and create draft commands, but only existing domain services can validate and execute authoritative mutations.

4. Proposed intelligence data contracts

These are new derived contracts that let all upgraded AI surfaces share the same evidence, context and action semantics. They should be Pydantic models / typed DTOs rather than free-form dictionaries.

Proposed intelligence data objects (derived - not new sources of truth)



Object	Minimum fields	Persistence
CaseContext	principal_scope, student/cohort ref, active filters, time window, domain summaries, missing_sources, pending_action_plan	Session store or signed client/server session; snapshot only when audit requires it
EvidenceClaim	claim_id, claim_type, source_type, source_id, locator, value_hash, as_of, permission_scope, status	Persist with AI artefact / audit reference
StudentTwinSnapshot	student_id, as_of, state summary, pressure_points, blockers, dependencies, model_signals	Derived cache optional; recomputable from source tables
InterventionPlan	plan_id, case_ref, rationale, actions[], owners, dependency graph, reassess_at, status	Persist draft + confirmed plan history
ScenarioResult	request, deterministic_diff, model_sensitivity, assumptions, missing_data	Short-lived; persist if user saves/uses in a decision
ChangeFinding	document versions, change_type, severity_for_review, source spans, reviewer disposition	Persist findings; never overwrite prior reviewed versions
PolicyProposal	source_policy_version, config_candidates, affected_query, simulation_result, approvals	Persist as governed change artefact

5. Groq gateway and model routing

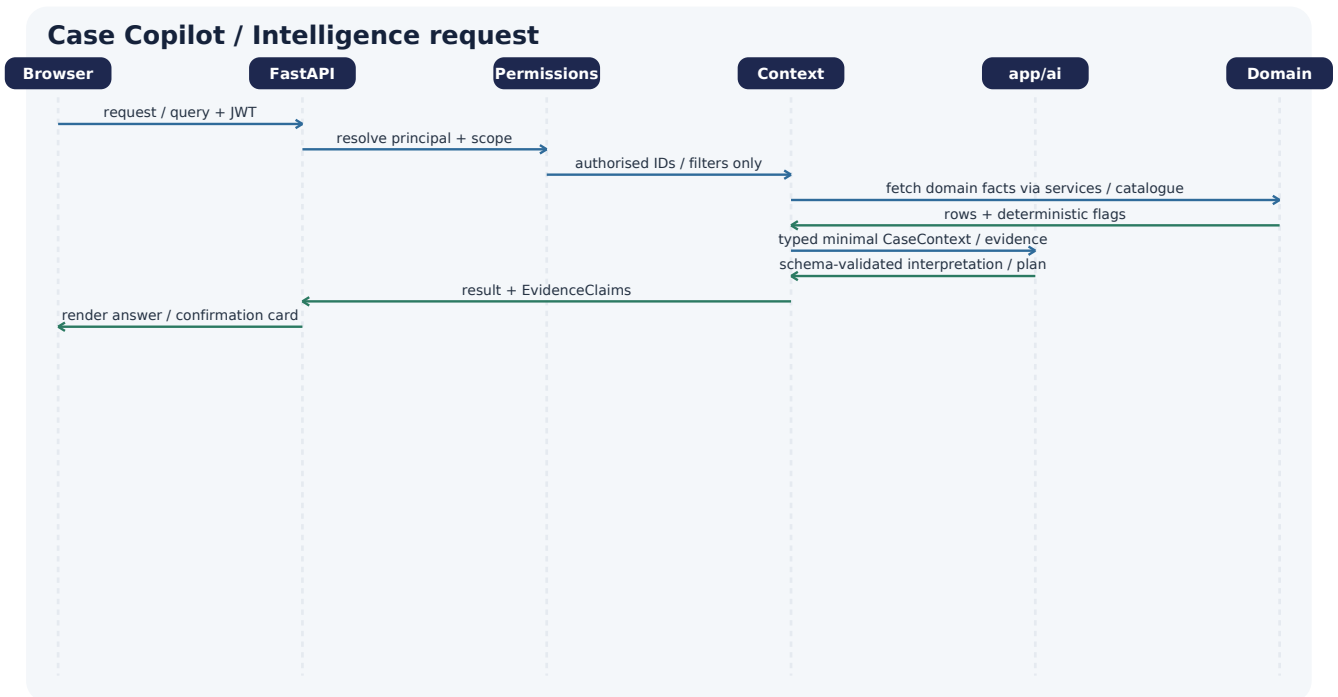
Your current inventory uses Groq gpt-oss-120b and gpt-oss-20b. Keep all external model calls behind app/ai so model choice, timeout, schema validation, retry and fallback policy stay centralised.

Call class	Default model strategy	Fallback / limits
classify	20b for intent edge cases, change type, intervention type	Deterministic rules / vocab classifier when available
rank	20b or 120b only when ranking quality materially improves	Deterministic priority score preserving mandatory urgent cases
read	120b for multi-document semantic comparison; 20b for short grounded extraction	Manual evidence view / deterministic extraction shell
narrate	20b for concise explanation and case summaries	Template narration from facts
complex plan	120b only for bounded multi-domain case plan with strict schema	Rule-based action templates + manual selection

- Centralise per-call timeout, retry count, token budget and model selection in app/ai.
- Return a typed result containing engine_used, fallback_path, prompt_template_version and latency_ms.
- Never retry an execution command because the LLM timed out; only retry model generation and keep idempotent action drafts.
- Add circuit-breaker behaviour so repeated Groq failures quickly fall back instead of adding latency to every request.
- If streaming is used for narrative, render deterministic evidence/results before the stream so the page remains useful without the model.

6. End-to-end request path and trust boundaries

A single request path should serve Ask PGR, the student Intelligence panel and intervention planning. The LLM never receives broad database access; it receives a prepared, authorised context and can only propose allow-listed operations.



For confirmed actions, start a separate execution request. Re-authorise the principal, re-read the target record, validate that preconditions still hold, then call the existing domain service. This prevents a stale AI plan from executing against changed state.

7. Case Context service

CaseContext is the shared memory/working-set layer behind Ask PGR, the Digital Twin and intervention planning. It should be typed and bounded - not an unstructured transcript memory.

Context field	Examples	Source
identity	student/person ID, institution, programme, study mode	Existing student/person services
time window	current term, next 90 days, user-selected range	Resolved intent / session state
academic state	upcoming milestones, conditions, panels, appeals	Progression service
supervision state	supervisors, last meeting, expected interval, open agreed actions	Supervision / meetings / intervention store
funding state	arrangements, coverage, integrity findings, payment state available in PGR	Funding service
thesis/documents	stage, corrections, selected document versions / locators	Thesis + object metadata
model signals	production predictions, model health, score history	Pattern Lab read service
workflow state	open tasks, owner, wait age, pending approvals	Workflow/task service
missing sources	e.g. Finance confirmation handler not available	Context Builder data-availability checks

8. Evidence Ledger service

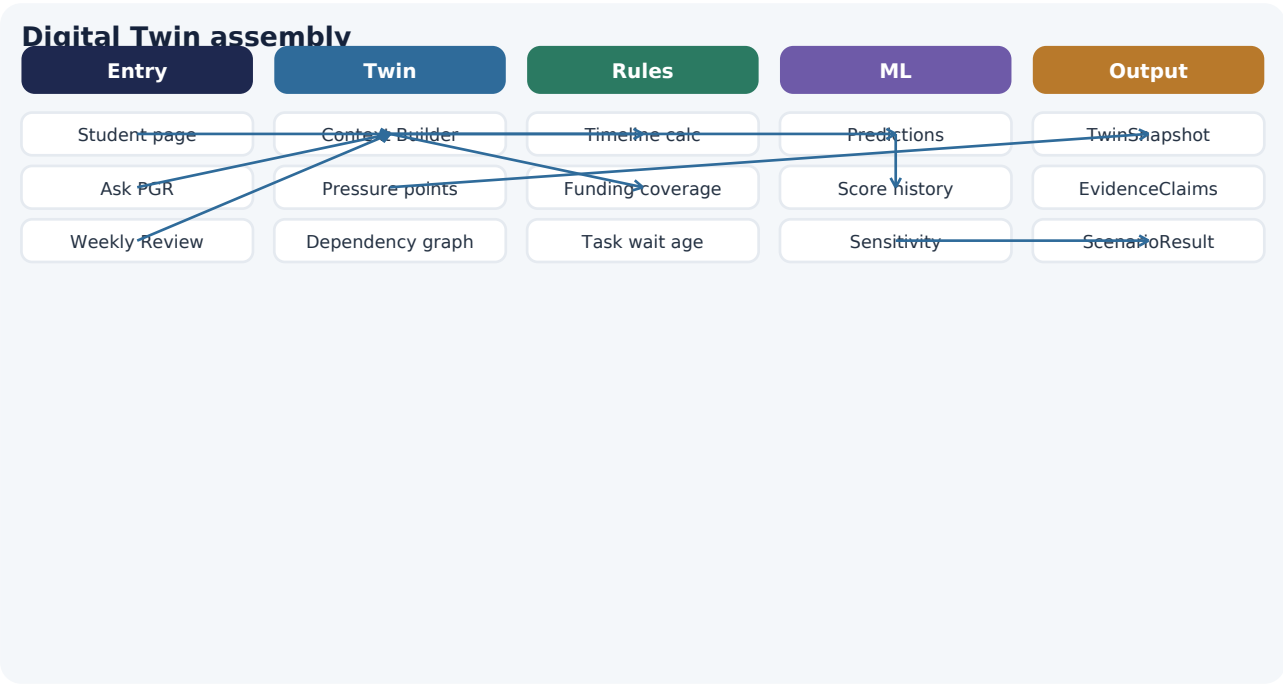
Use one evidence envelope across LLM answers, rankings, document findings, scenarios and intervention plans. This is the foundation for your "why this answer" panel to become an enterprise-grade evidence view.

Field	Purpose
claim_id / artefact_id	Stable link from narrative sentence / card to its evidence.
claim_type	deterministic_fact model_signal document_finding user_context missing_data.
source_type + source_id	Student row, milestone, meeting, funding arrangement, workflow, model version, document version.
locator	Field path for structured rows or page/section/span hash for documents.
as_of / freshness	When the evidence was read; allows stale-data warnings.
permission_scope	Principal/institution context under which the evidence was visible.
status	verified inferred_for_review missing superseded.
value_hash	Detect whether the underlying source changed after an AI artefact was created.

When the user reopens an old AI artefact, compare value_hash / updated_at to current source state. If evidence changed, mark the artefact "out of date" and offer to regenerate rather than silently presenting stale reasoning.

9. PGR Digital Twin - backend design

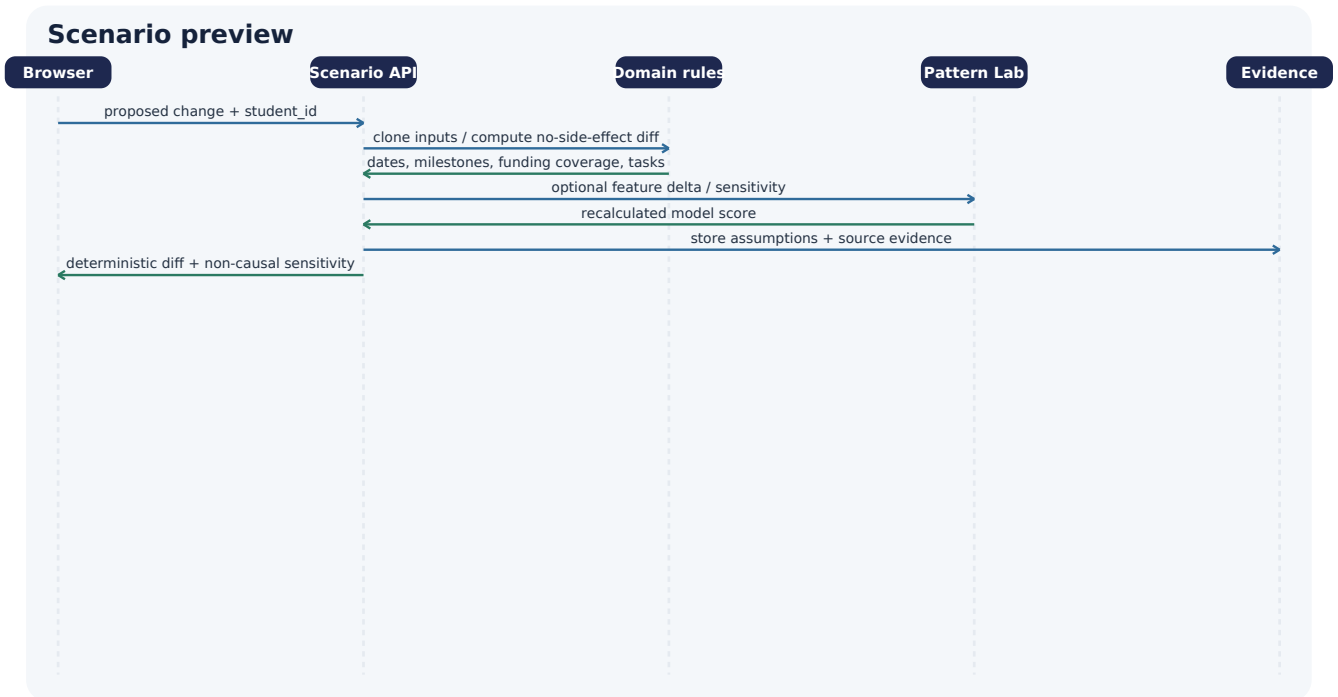
The twin is a derived read model assembled from existing authoritative services. Do not create a second student state table that must be manually kept in sync.



Subcomponent	Implementation detail
Pressure-point score	Deterministic weighted ordering of upcoming deadlines, open blockers, wait age and governed model signals. Keep weights configurable/auditable.
Dependency graph	Explicit edges such as "student chapter update depends on supervisor dataset" and "review decision depends on panel".
Snapshot strategy	Generate on demand; optionally cache in Postgres/materialised view for dashboard scale. Invalidate or version by domain updated_at/event sequence.
Scenario entry	Pass proposed lifecycle change to side-effect-free scenario service; do not mutate the twin/source record.

10. Scenario Engine - separate deterministic impact from model sensitivity

Create a dedicated preview API. It should calculate what the configured rules say will change, then optionally calculate how a production model score would respond to changed feature inputs. These two outputs must stay separate all the way to the frontend.



- No notifications, workflow tasks, outbox events, audit mutations or external calls from preview calculations.
- Prefer pure domain calculation functions. If existing services cannot be called without side effects, extract the calculation logic before building simulation.
- Expose assumptions explicitly: proposed start/end, study mode, whether funding is assumed unchanged, which milestones are already decided.
- If the user chooses "Start request from this scenario", copy only the user-confirmed inputs into the real lifecycle request and run normal validation again.

11. AI Intervention Engine - planner and execution boundary

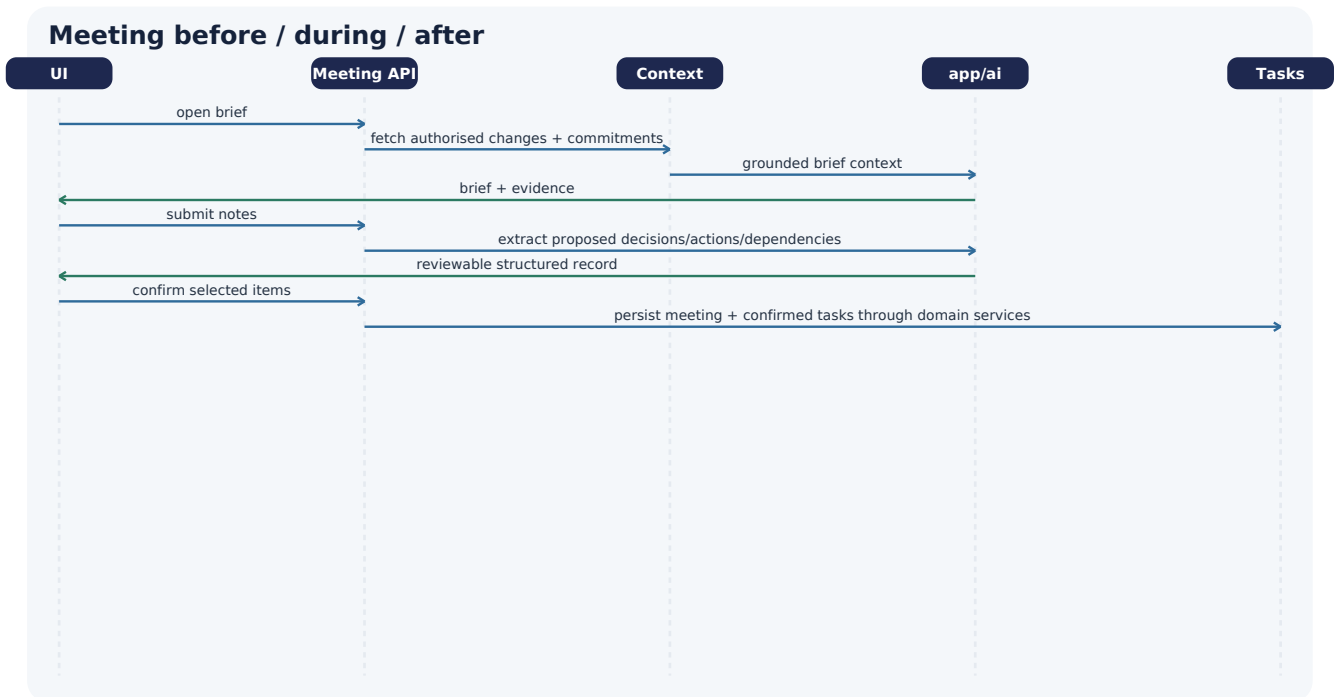
Intervention planning should operate on bounded action templates. The LLM may select and explain templates, but it should not invent arbitrary API calls or owners.



Action-template rule	Example
Allow-listed type	create_task, prepare_meeting_brief, request_funding_review, open_review_evidence_check, schedule_reassessment
Owner resolver	Resolve from existing supervisor relationship, workflow role or explicit administrator selection
Due date source	Policy/configuration, existing deadline, user choice or "not specified" - never LLM-invented
Idempotency key	case_ref + action_type + target + source_event to prevent duplicate confirmed/retried actions
Execution precondition	Target state and permission must be revalidated immediately before domain write

12. Supervision Copilot - meeting lifecycle backend

Extend the existing Meeting Brief service with structured commitments rather than replacing it. Pasted notes are the safest first input; audio transcription can be added later behind separate consent/data-handling controls.



Proposed commitment fields: `source_meeting_id`, `text`, `owner_person_or_role`, `dependency_ids[]`, `due_at` nullable, `status`, `evidence/document link`, `confirmed_by`, `confirmed_at`. A null due date is a valid state and should remain null when the meeting did not specify one.

13. Engagement Trajectory - derived signal backend

Keep the existing relationship-signal fallback, but store the inputs and output over time so the badge becomes a trajectory. The deeper view should be reconstructable from deterministic interaction events.

Signal input	Source	Treatment
Meeting recency / expected interval	Supervision + institution setting	Deterministic
Meeting frequency trend	Meeting history	Deterministic
Confirmed supervision actions	Intervention / task data	Deterministic
Approved context override	Authorised user entry	Deterministic context
Narrative label	app/ai classify/narrate	LLM optional; fallback always available

14. Composer Investigator - investigation graph backend

The fixed function catalogue remains the safety boundary. Extend each catalogue function so it can expose dimensions, drill paths and contributing record IDs in a typed result, then let the LLM choose the next legal investigative step.

Function result field	Purpose
metric	Primary value / series used for chart
comparison	Prior period / benchmark result
dimensions	Department, stage, programme, funding source, etc.
drill_paths	Allow-listed next functions + required parameters
contributing_ids	Authorised record IDs that explain an aggregate
evidence_claims	Source metadata / as-of time for narrative
saveable_definition	Function IDs + parameters for scheduled monitor

This makes root-cause exploration deterministic at the data layer. The LLM decides which legal function to call next and narrates the returned numbers; it never invents a query plan outside the catalogue.

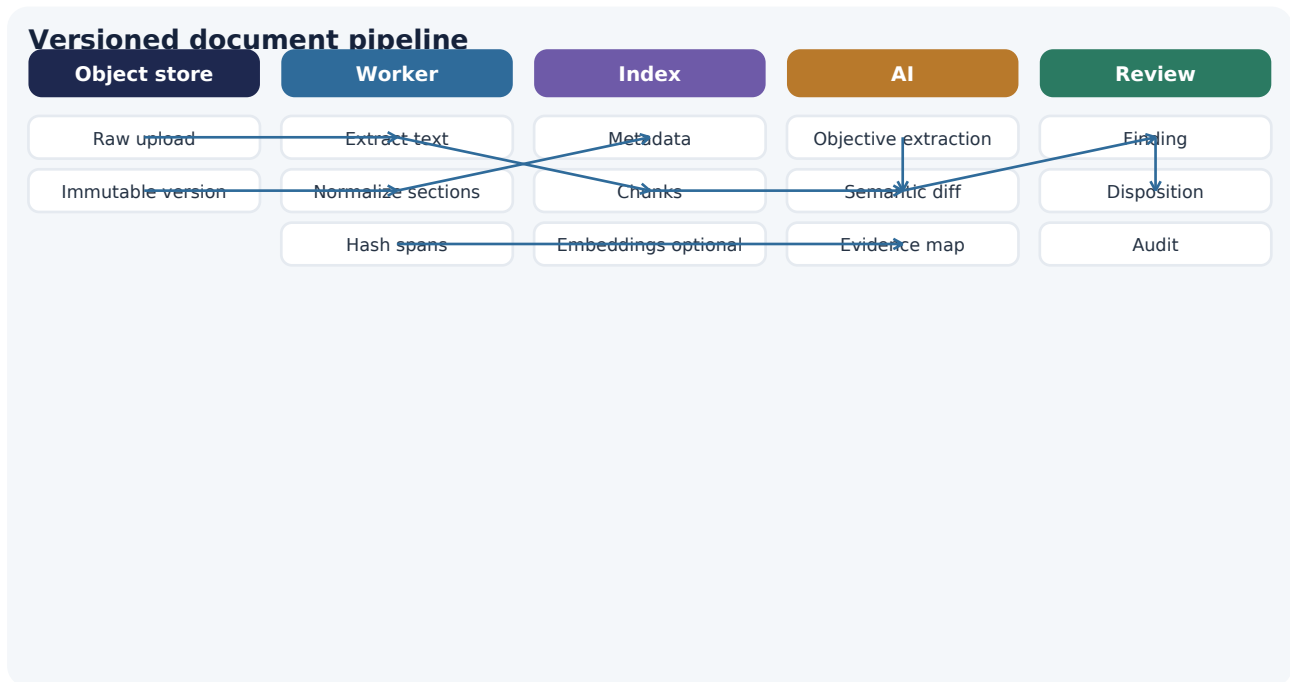
15. Pattern Lab upgrades - prediction history and sensitivity

Do not modify the governance lifecycle. Add data products around production predictions so the UI can explain how a student score moved over time and safely explore sensitivity.

New store / service	Fields / behaviour
PredictionSnapshot	student_id, target, model_version, predicted_at, probability, threshold, model_health_ref
DriverSnapshot	method, feature, direction, delta_pp_or_sensitivity, baseline/input value, generated_at
ModelVersionBoundary	Mark when a new production version changes score comparability
SensitivityRunner	Apply bounded feature variants to the exact production pipeline without persisting the changed student features
HealthGate	If drift/calibration/AUC health is poor, add warning or disable intervention-oriented narrative

16. Document Intelligence foundation

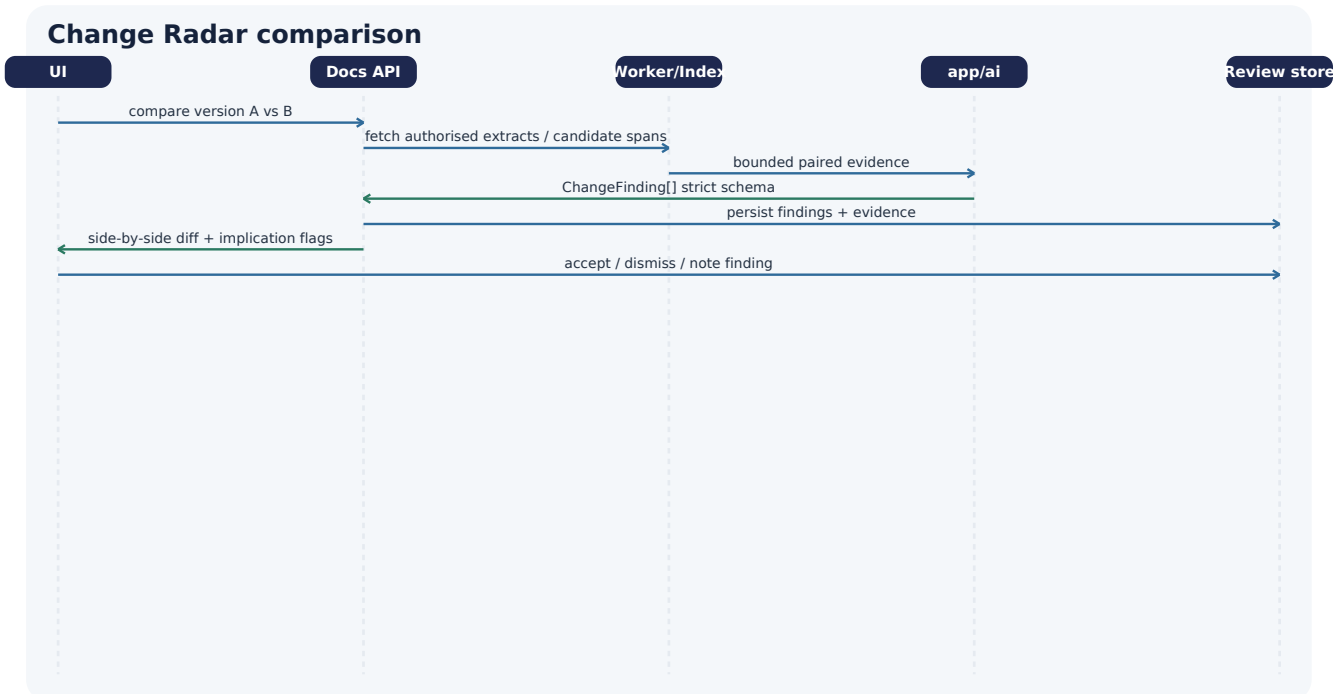
Research Change Radar, review evidence mapping and future correction comparison all depend on a reliable document pipeline. Keep raw documents in the existing object store; store extracted/version metadata in PostgreSQL.



- Suggested metadata: document_id, version_id, object_key, content_hash, mime_type, uploaded_at, uploaded_by, extraction_status, extractor_version, page_count.
- Suggested chunk locator: page_number, section_heading, char_start/end, span_hash; store enough to reopen the exact passage.
- Do not make embeddings the only evidence pointer. Embeddings help retrieval; exact locators support review/audit.
- Re-extraction should create a new extraction revision if the parser changes, preserving the basis of prior reviewed findings.

17. Research Change Radar - comparison pipeline

The LLM compares structured extracts and candidate matching passages, but every finding must retain both source sides and a human disposition.



Change types should be an allow-list such as `objective_added/removed/modified`, `method_changed`, `population_changed`, `sample_size_changed`, `scope_changed`, `timeline_changed`, `limitation_added`, `ethics_relevant_change_for_review`. The last is a review flag, not an ethics decision.

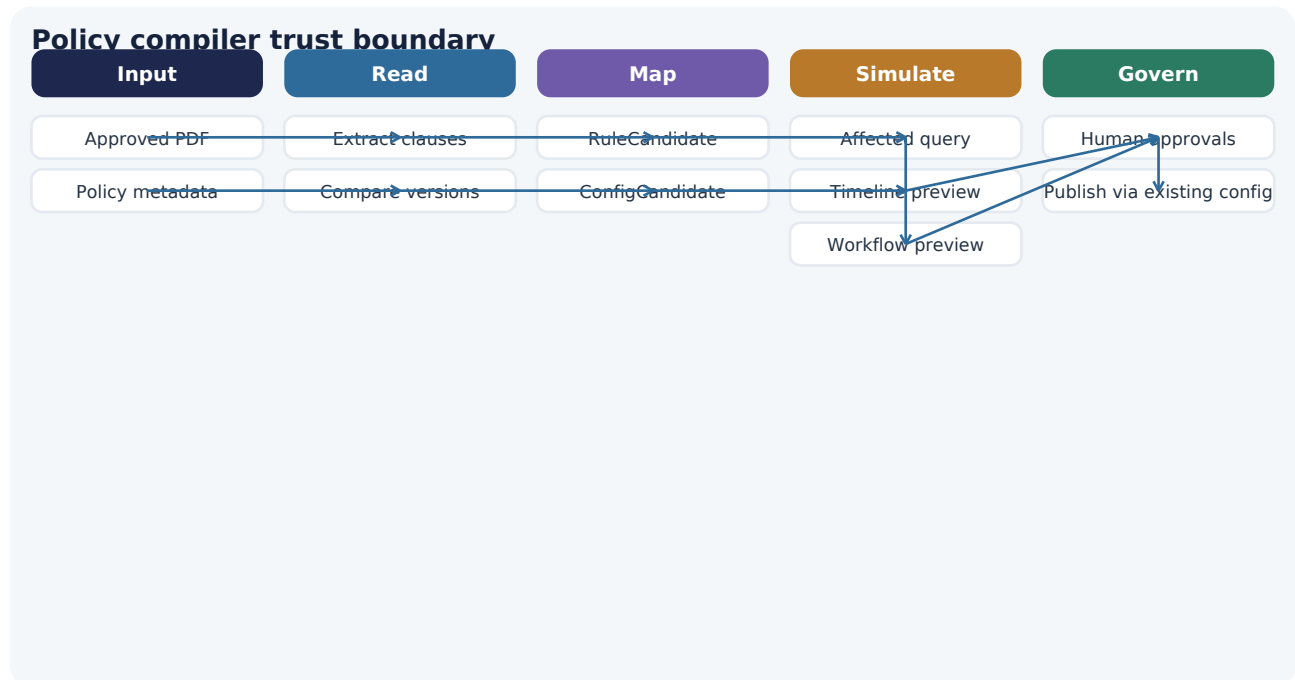
18. Institutional Memory - precedent retrieval design

This service should not be a generic vector database over entire case files. First build a governed structural feature representation and permission filters, then use similarity only within the allowed candidate set.

Step	Implementation
Candidate filter	Same institution + authorised case class + compatible programme/request type/date policy window.
Structured similarity	Compare lifecycle stage, study mode, timing, funding coverage, milestone context, evidence requested and workflow path.
Optional semantic similarity	Use de-identified case summaries / selected notes only after permission and minimisation.
Narration	Summarise common handling/evidence and show variance; never infer "therefore approve".
Audit	Log which cases contributed to the summary and whether the user opened them.

19. Policy Compiler - controlled configuration proposal

The policy compiler has the highest governance risk, so the architecture must separate document understanding from executable configuration. Only approved policy versions and allow-listed configuration targets can reach the proposal stage.



Do not allow the LLM to emit arbitrary Python/SQL/rule expressions. Map only to a constrained schema representing existing settings and explicitly supported workflow/rule configuration. Unknown clauses become "manual implementation required" rather than executable guesses.

20. Worker integration and scheduled intelligence

The existing APScheduler worker is already responsible for outbox, jobs and notifications. Use it for expensive/non-interactive AI work and material-change monitors, but keep interactive reads in FastAPI when latency is acceptable.

Work item	Where	Trigger / schedule
Document extraction / indexing	Worker	On upload / new version
Research Change Radar precompute	Worker	After extraction or reviewer request
Weekly Review ranking	Worker or existing scheduled path	Monday / configured cadence
Proactive systemic alerts	Worker	Daily / weekly material-change checks
Intervention reassessment	Worker	reassess_at timestamp or source event
Twin snapshot cache refresh	Worker optional	Domain event / periodic refresh if materialised
Policy impact simulation	Worker for large cohorts	User request -> job -> result notification

Preserve the existing outbox model for external side effects. AI jobs should not call partner endpoints directly; they should invoke domain services that emit normal transactional outbox events only after a human-confirmed action.

21. PostgreSQL and read-replica strategy

Most new intelligence state is derived metadata, evidence and history rather than large model blobs. Keep the operational write path on the primary; route eligible analytical reads to the replica as the existing architecture already intends.

Data class	Primary / replica approach
CaseContext session state	Primary or ephemeral store; small and user-scoped
EvidenceClaim / AI artefacts	Primary writes; historical reads can use replica
Prediction / driver snapshots	Primary append; trend reads can use replica
Intervention plans/outcomes	Primary because they feed workflow decisions
Document metadata / findings	Primary append; comparison/search reads may use replica
Composer aggregates	Read replica preferred where freshness permits
Embeddings (if in Postgres)	Replica-readable; ensure extension/index strategy is supported in target environment

22. Proposed API surface - additive, not disruptive

These endpoint names are recommendations, not claims about current routes. They keep intelligence APIs explicit while reusing current domain services for execution.

Proposed endpoint	Purpose	Mutation ?
GET /api/v1/intelligence/students/{id}	StudentTwinSnapshot + current intelligence summary	No
POST /api/v1/intelligence/scenarios	Side-effect-free deterministic + optional model sensitivity preview	No
POST /api/v1/intelligence/interventions/plan	Generate typed InterventionPlan	Draft only
POST /api/v1/intelligence/interventions/{plan}/confirm	Re-authorise and dispatch selected actions via domain services	Yes - confirmed
GET /api/v1/intelligence/evidence/{artefact}	Evidence Ledger for an answer/card/finding	No
POST /api/v1/intelligence/documents/compare	Create/queue Change Radar comparison	Job / no domain mutation
GET /api/v1/intelligence/cases/similar	Permission-filtered Institutional Memory candidates	No
POST /api/v1/intelligence/policy/proposals	Create governed policy mapping proposal	Draft only

23. Security and privacy controls

Preserve the current strong row-scoping design and extend it consistently to documents, caches, evidence and embeddings. The most important rule is that permissions constrain retrieval before the prompt, not through instructions inside the prompt.

Control	Required design
Row / tenant scoping	Every query includes institution/principal scope resolved from JWT/roles. Out-of-scope IDs return not found as today.
Document scoping	Authorise document/version before extraction retrieval, semantic search and evidence reopening.
Prompt injection	Uploaded research/policy text is untrusted data. Never let document text alter tools, system instructions or allowed actions.
Data minimisation	Prompt only required fields and passages. Pseudonymise names when identity does not affect the task.
Egress control	Allow Groq only via approved HTTPS destination / gateway and record the production egress policy change.
Audit	Log model, template version, evidence IDs, fallback path, action confirmation, user override and request ID.
Deletion / subject access	Ensure derived AI artefacts/evidence references participate in institutional retention/erasure processes before go-live.

24. Model-off, failure and idempotency architecture

The existing model-off proof test is a strong design asset. Extend it so every upgraded feature has an explicit deterministic/manual degradation mode and action execution remains idempotent.

Feature	When Groq is unavailable
Case Copilot	Fuzzy/router + deterministic cohort/domain responses; manual action-template picker; case filters preserved.
Weekly Intervention Queue	Deterministic priority + rule-based intervention mapping for known cases.
Supervision Copilot	Existing Meeting Brief fallback + manual commitments form.
Digital Twin	All deterministic state/pressure points render; narrative and complex classification degrade.
Pattern Lab	Predictions continue; narrative / sensitivity explanation template-based.
Change Radar	Document versions remain viewable side-by-side; semantic findings unavailable or queued for retry.
Policy Compiler	Source/version comparison and manual mapping workflow remains; AI proposals unavailable.

Add idempotency keys to confirmed multi-action plans and worker jobs. A Groq retry must never duplicate tasks, approvals, notifications or outbox events.

25. Observability and AI-specific SLOs

The existing architecture already plans OTel / Grafana or Datadog and Sentry. Add AI telemetry to the same observability stack rather than a separate black box.

Metric	Target / use
Interactive factual shell latency	Keep deterministic record/evidence render within existing API/dashboard expectations where possible.
LLM latency p50/p95	Track by call class/model; UI should tolerate narrative arriving after facts.
Fallback rate	Monitor by feature; spikes indicate Groq/provider or prompt/schema problems.
Schema failure / repair rate	Should be near zero for production templates; alert on sustained increase.
Evidence coverage	% AI artefacts whose factual claims have complete EvidenceClaims.
User edit / override rate	Quality signal for plans, summaries and Change Radar findings.
Cost	Tokens / request, cost / active PGR / month, batch-job cost by feature.
Model health	Pattern Lab PSI, matured AUC, calibration / existing governance signals.

26. Scaling and deployment alignment

The current architecture sizes application, worker and database tiers by active PGR population. AI introduces two new scaling patterns: external LLM concurrency and document-processing throughput. Scale those separately while leaving core API/database rules intact.

Pressure	Recommended response
Interactive LLM concurrency	Per-user and global concurrency limits; preserve current /assistant rate limit and add feature budgets.
Document backlog	Worker queue / job table with bounded concurrency; add worker before API pods if extraction queue grows.
Composer / cohort analytics	Prefer read replica and pre-aggregated/materialised views where already planned.
Evidence / history growth	Append-only compact metadata; archive according to institutional retention without breaking source links.
Groq outage / rate limit	Circuit breaker -> deterministic fallback; never saturate API workers waiting on model calls.
Large policy simulation	Background job with progress/result notification rather than long HTTP request.

Update production egress/WAF/network documentation and load tests to include the external AI dependency. The core platform should still meet its baseline read/write SLOs when the AI provider is slow or unavailable.

27. Test strategy

Extend the existing 224-test baseline and model-off proof philosophy with contract, permission, prompt-security and no-side-effect tests. The numbers below are test categories, not a required exact test count.

Test family	Examples
Contract tests	All app/ai and intelligence schemas reject missing/extra/invalid fields; deterministic fallback has same outer contract.
Permission tests	Out-of-scope student/document never enters CaseContext, embeddings, evidence or model prompt.
No-side-effect scenario tests	Scenario preview produces zero domain mutations, notifications and outbox events.
Confirmation tests	Staged action cannot execute without current user confirmation; state drift causes revalidation failure.
Idempotency tests	Retrying confirm/job does not duplicate task/outbox/action.
Model-off E2E	Every upgraded surface renders valid deterministic/manual state with ASSISTANT_LLM_ENABLED off.
Prompt-injection tests	Malicious document text cannot change allowed tool/action schema.
Evidence integrity	Every factual card links to current authorised source; stale hashes are detected.
ML tests	Score history, model-version boundaries and sensitivity use the exact governed production pipeline.

28. Recommended module map and ownership

Keep the modular monolith understandable by giving each intelligence concern a narrow package and explicit dependency direction.

backend/app/intelligence/

context/ - CaseContext builders and session working set

evidence/ - EvidenceClaim schema, provenance, freshness checks

twin/ - StudentTwinSnapshot, pressure-point and dependency assembly

scenario/ - deterministic previews + Pattern Lab sensitivity adapter

interventions/ - templates, planner, staging, idempotency, outcome snapshots

supervision_ai/ - brief augmentation and commitment extraction

documents/ - extraction metadata, locators, comparison orchestration

memory/ - governed case features and similarity

policy/ - approved policy ingestion, mapping proposals and simulation

investigation/ - Composer drill graph / monitor definitions

observability/ - AI telemetry, fallback, cost and quality metrics

Dependency rule: app/intelligence -> app/ai + read/query/domain interfaces. Domain modules must not depend on LLM output to perform core business rules.

29. Delivery plan and architecture gates

Build the shared contracts and trust layer first. Do not start Policy Compiler or broad document semantics until row/document permissions, evidence locators and model-off behaviour are proven.

Phase	Backend deliverables	Gate to proceed
1 - Foundations	EvidenceClaim, CaseContext, action-plan schema, prediction history, AI telemetry	Permission tests + model-off E2E + evidence UI working
2 - Existing feature upgrades	Case Copilot, Intervention planner, Supervision commitments, Engagement trajectory	Confirmed actions idempotent; reassessment works
3 - Digital Twin + scenarios	TwinSnapshot, pressure points, deterministic preview, Pattern Lab sensitivity adapter	No-side-effect tests; scenario assumptions visible
4 - Document intelligence	Extraction revisions, locators, Change Radar	Source passage accuracy and prompt-injection tests
5 - Institutional Memory	Governed case features + similarity + precedent summaries	Privacy/permission review; outcome-bias language tested
6 - Policy Compiler	Approved policy workflow, allow-listed mapping, cohort simulation	Registry sign-off; config rollback and impact tests

30. Architecture assumptions and unresolved items

This architecture is intentionally aligned to the supplied documents, but several production details are not established by those sources and should remain explicit decisions.

- The companion PGR_Platform_Critical_Gaps.pdf referenced in both supplied documents was not provided. Review it before committing to production sequencing.
- The current AI inventory proves Groq is in use, but the supplied network architecture does not show AI-provider egress. Production network policy/documentation must be updated.
- The supplied architecture shows Finance inbound handler as TBD. AI must not claim payment confirmation beyond the PGR-side state until that integration exists.
- The documents show object storage for theses/letters/IDs but do not establish a versioned extraction/indexing layer; this is new work required for Research Change Radar.
- The documents show institutional settings and statutory profiles but not a general executable policy-rule DSL. Policy Compiler should initially target only explicitly supported/allow-listed configuration.
- SSO is listed as later. AI permissioning should continue to rely on the same principal/role model as the rest of the app and migrate cleanly when OIDC is introduced.